

ALGORITHMS AND DATA STRUCTURES IN JAVA

ANVESH RACHAKONDA

Task 1: (30 points)

Part 1 – Data Structure Selection Reasoning

You are given the following scenarios. For each, choose the most appropriate Java data structure and justify your choice

1. Recent Actions in a Text Editor – You need to store and undo the most recent operations performed by the user.

A stack is the data structure for storing recent actions in a text editor . It follows the Last In First out principle meaning the most recently added is removed first . This matches the behavior of an undo operation because the latest user action should be undone before earlier actions.

2. Customer Service Call Center – Customers are served in the order they call.

A queue is the data structure for a customer service call centre . It follows the First In , First Out (FIFO) Principle , meaning the first customer who enters the queue is the first customer to be served. This matches the requirement that customers should be served in the order they call . A queue allows customers to wait fairly until it is their turn to receive service.

3. Leaderboard in a Game – Players are ranked by their score, highest first.

A priority queue is suitable because players need to be ranked by their scores . It allows the highest priority score to be accessed first . A reverse ordering can be used so the highest score comes first.

4. Unique Email Addresses – You need to store a collection of email addresses without duplicates

A HashSet is suitable for storing unique email addresses . It does not allow duplicate values . It also provides fast operations for adding and checking email addresses.

5. Student Grades Lookup – You need to quickly find a student's grade given their student ID

A HashMap is suitable for storing student ID's and their grading . the student ID can be used as the key and the grade as the value . It allows fast lookup of a grade using the student ID.

6. Playlist of Songs – You need to store songs in order and allow random access by index

An ArrayList is suitable for storing songs in order . It allows elements to be accessed quickly using an index . It can also grow dynamically when more songs are added.

Part 2 – OOP Concept Identification

Below is a short Java code snippet. Identify which OOP concept(s) are demonstrated in each marked section and explain your reasoning.

Tasks:

- 1. For [A], identify the OOP concept(s) and explain why.**
- 2. For [B], identify the OOP concept(s) and explain why.**
- 3. For [C], identify the OOP concept(s) and explain why.**

```
// [A]
public abstract class Shape {
    private String color;

    public Shape(String color) {
        this.color = color;
    }

    public String getColor() { return color; }
    public void setColor(String color) { this.color
= color; }

    public abstract double area();
}

// [B]
public class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}

// [C]
public class ShapePrinter {
    public void printArea(Shape s) {
        System.out.println("Area: " + s.area());
    }
}
```

For A:

Abstraction and encapsulation : The shape class demonstrates abstraction because it is an abstract class with an abstract area() method. It demonstrates encapsulation because the color variable is private and accessed through getter and setter methods.

For B:

Inheritance and Polymorphism : The circle class demonstrates inheritance because it extends the shape class > It demonstrates polymorphism by overriding the area() method with its own implementation.

For C:

Polymorphism : The shapePrinter class demonstrates polymorphism because it accepts a shape objects and call its area () method. Different subclasses of shape can provide their own implementation of area().

- **Why is it important to choose the right data structure for a problem?**
- **How do OOP concepts help in making code more maintainable and reusable?**

Choosing the right data structure helps a program store and access data efficiently . Different data structure are designed for different types of problems and operations . Using the correct structure can improve the performance of a program . OOP concepts such as encapsulation , inheritance, polymorphism , and abstraction help organize code . They make programs easier to maintain , reuse , and modify . These concepts also help reduce code duplication and make large programs easier to manage.

Task 2 (30 points)**Part 1:**

Below are five scenarios. For each, you must:

- 1. Identify the most likely time complexity (choose from $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$).**
- 2. Justify your choice in 3–5 sentences, explaining the reasoning and trade-offs**

Scenarios:**1. Looking up a value in a HashMap by key.**

- **$O(1)$** – Hashmap lookup by key is generally $O(1)$. It uses the key's hash value to find the location of the value . Therefore, the lookup usually takes constant time regardless of the number of elements.

2. Performing a binary search on a sorted array.

- **$O(\log n)$ — Logarithmic Time:** Binary search repeatedly divides the sorted array into two halves. Each step eliminates half of the remaining elements. Therefore, the number of steps grows logarithmically as the array size increases.

3. Printing all elements in a list of size n.

- **$O(n)$ — Linear Time:** Printing all elements requires visiting each element in the list. If there are n elements, the program performs n operations. Therefore, the time complexity is $O(n)$.

4. Sorting an array using Merge Sort.

- **$O(n \log n)$ — Linearithmic Time:** Merge Sort divides the array into smaller halves and recursively sorts them. The array is divided into about $\log n$ levels, and each level processes n elements. Therefore, the overall time complexity is $O(n \log n)$.

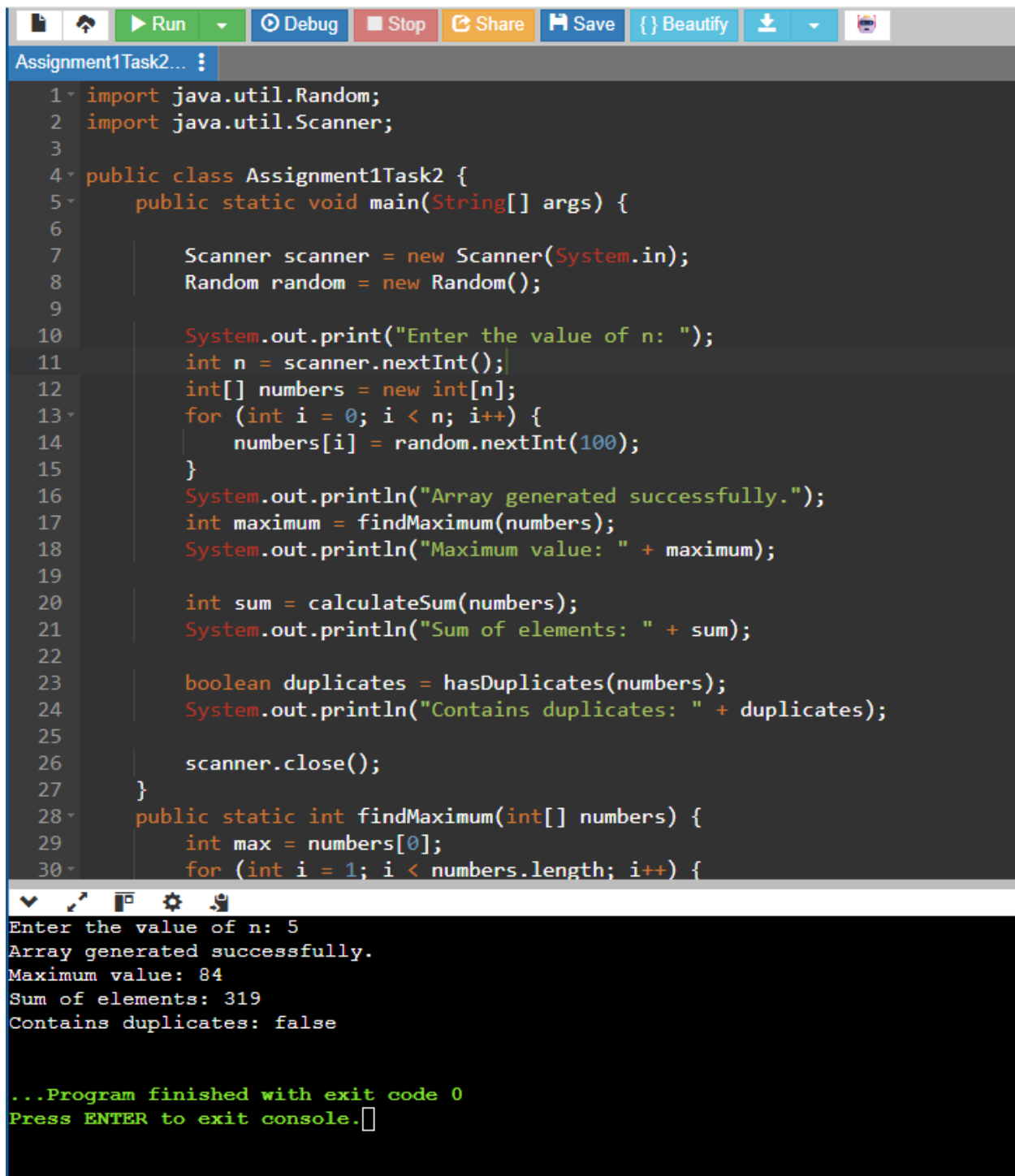
5. Checking if a list contains any duplicate elements using a nested loop

- **$O(n^2)$ — Quadratic Time:** The nested loop compares elements with each other to find duplicates. As the number of elements increases, the number of comparisons grows approximately as n^2 . Therefore, the time complexity is $O(n^2)$.

Part 2:

Problem Statement: Write a Java program that:

1. Reads an integer n from the user.
2. Generates an array of n random integers.
3. Finds and prints:
 - o The maximum value in the array.
 - o The sum of all elements.
 - o Whether the array contains any duplicates



```
1 import java.util.Random;
2 import java.util.Scanner;
3
4 public class Assignment1Task2 {
5     public static void main(String[] args) {
6
7         Scanner scanner = new Scanner(System.in);
8         Random random = new Random();
9
10        System.out.print("Enter the value of n: ");
11        int n = scanner.nextInt();
12        int[] numbers = new int[n];
13        for (int i = 0; i < n; i++) {
14            numbers[i] = random.nextInt(100);
15        }
16        System.out.println("Array generated successfully.");
17        int maximum = findMaximum(numbers);
18        System.out.println("Maximum value: " + maximum);
19
20        int sum = calculateSum(numbers);
21        System.out.println("Sum of elements: " + sum);
22
23        boolean duplicates = hasDuplicates(numbers);
24        System.out.println("Contains duplicates: " + duplicates);
25
26        scanner.close();
27    }
28    public static int findMaximum(int[] numbers) {
29        int max = numbers[0];
30        for (int i = 1; i < numbers.length; i++) {
```

```
Enter the value of n: 5
Array generated successfully.
Maximum value: 84
Sum of elements: 319
Contains duplicates: false

...Program finished with exit code 0
Press ENTER to exit console.
```

Requirements:

- Implement each of the three operations in separate methods.
- After writing the code, analyze each method:
 - State its time complexity
 - Explain why.

1. findMaximum()

Time Complexity: $O(n)$

The method checks each element in the array once to find the maximum value. Therefore, the number of operations increases linearly with the size of the array.

2. calculateSum()

Time Complexity: $O(n)$

The method visits every element once and adds it to the sum. Therefore, its time complexity is $O(n)$.

3. hasDuplicates()

Time Complexity: $O(n^2)$

The method uses two nested loops to compare elements with each other. As the array size increases, the number of comparisons grows quadratically. Therefore, its time complexity is $O(n^2)$.

Task 3 (40 points)

Part 1:

Instructions: You are given three sorting techniques and two searching techniques:

- $O(n^2)$ Sorts: Bubble Sort, Insertion Sort
- Non- $O(n^2)$ Sorts: Merge Sort (recursive), Quick Sort (recursive)
- Search Techniques: Linear Search, Binary Search (recursive)

For each technique:

1. Describe how it works in your own words (max 150 words each).
2. Write the pseudo-code (language-agnostic, but clear and structured).
3. Calculate and discuss performance:
 - Best-case, worst-case, and average-case time complexity.
 - Space complexity (including recursion stack if applicable).
4. For Merge Sort, Quick Sort, and Binary Search:
 - Clearly identify the divide and conquer steps.
 - Explain the base case and recursive case.

Deliverable: A well-structured document or table summarizing:

- Algorithm name
- Description
- Pseudo-code
- Time complexities

1. Algorithm Name : Bubble Sort

2. Description:

Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements in a list. If two adjacent elements are in the wrong order, they are swapped. After each pass, the largest unsorted element moves toward the end of the list. The process continues until all elements are sorted. Bubble Sort is easy to

understand and implement, but it is inefficient for large datasets because it can require many comparisons and swaps.

This is well below the 150-word maximum.

3. Pseudocode:

BubbleSort(array)

 for i = 0 to n - 1

 for j = 0 to n - i - 2

 if array[j] > array[j + 1]

 swap array[j] and array[j + 1]

4. Time Complexity:

CASE	COMPLEXITY
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

5. Space Complexity: $O(1)$

Bubble Sort sorts the elements in the original array and does not require additional storage.

1. Algorithm Name : Insertion Sort

2. Description

Insertion Sort builds the sorted list one element at a time. It takes the next element and compares it with the elements before it. If necessary, larger elements are shifted to the right to make space for the current element. The current element is then inserted into its correct position. This process continues until all elements are sorted. Insertion Sort is simple and works well for small or nearly sorted lists, but it can be inefficient for large unsorted lists.

3. Pseudocode

InsertionSort(array)

 for i = 1 to n - 1

 key = array[i]

 j = i - 1

 while j >= 0 AND array[j] > key

```
array[j + 1] = array[j]
j = j - 1
array[j + 1] = key
```

4. Time Complexity:

CASE	COMPLEXITY
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

5. Space Complexity: $O(1)$

Insertion Sort sorts **the array in place and uses only a small amount of additional memory.**

1. Algorithm Name: Merge Sort

2. Description:

Merge Sort is a sorting algorithm that uses the divide-and-conquer approach. It divides the array into two smaller halves and recursively sorts each half. Once the two halves are sorted, they are merged together in the correct order. This process continues until each part contains only one element, which is already sorted. The sorted parts are then combined to produce the final sorted array. Merge Sort provides consistent performance for large datasets and has a time complexity of $O(n \log n)$.

3. Pseudocode:

```
MergeSort(array)
```

```
    if array has one element
        return
```

```
    find the middle of the array
    left = first half
    right = second half
```

```
    MergeSort(left)
```

MergeSort(right)

Merge left and right in sorted order

Merge operation

Merge(left, right)

create an empty result

while both left and right have elements

if left element $\leq$ right element

add left element to result

else

add right element to result

add remaining elements from left

add remaining elements from right

return result

4. Divide-and-Conquer

Divide: Split the array into two halves.

Conquer: Recursively sort each half.

Combine: Merge the two sorted halves into one sorted array.

5. Base Case: The base case occurs when the array contains one element or zero elements. It is already sorted, so no further splitting is needed.

6. Recursive Case: If the array contains more than one element, it is divided into two halves. Merge Sort is recursively called on both halves, and the sorted halves are then merged.

7. Time Complexity:

CASE	COMPLEXITY
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

8. Space Complexity: $O(n)$

Merge Sort requires additional memory to store the temporary arrays/elements during the merging process. The recursive calls also use stack space.

1. Algorithm Name: Quick Sort

2. Description

Quick Sort is a sorting algorithm that uses the divide-and-conquer approach. It selects an element as a pivot and rearranges the array so that smaller elements are placed before the pivot and larger elements are placed after it. The algorithm then recursively sorts the elements on both sides of the pivot. This process continues until the subarrays contain zero or one element. Quick Sort can be very efficient in practice, although its performance depends on how the pivot is selected.

3. Pseudocode:

QuickSort(array, low, high)

 if low < high

 pivotIndex = Partition(array, low, high)

 QuickSort(array, low, pivotIndex - 1)

 QuickSort(array, pivotIndex + 1, high)

Partition

Partition(array, low, high)

 pivot = array[high]

 i = low - 1

 for j = low to high - 1

 if array[j] <= pivot

 i = i + 1

 swap array[i] and array[j]

 swap array[i + 1] and array[high]

 return i + 1

4. Divide-and-Conquer:

Divide: Choose a pivot and partition the array around it.

Conquer: Recursively sort the elements before and after the pivot.

Combine: The sorted partitions together form the sorted array.

5. Base Case: The base case occurs when the subarray has zero or one element. It is already sorted.

6. Recursive Case: When the subarray has more than one element, Quick Sort partitions it around a pivot and recursively sorts the two resulting subarrays.

7. Time Complexity:

CASE	COMPLEXITY
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$

8. Space Complexity:

$O(\log n)$ average case for the recursion stack and $O(n)$ worst case when the recursion becomes highly unbalanced.

1. Algorithm Name: Quick Sort

2. Description:

Quick Sort is a sorting algorithm that uses the divide-and-conquer approach. It selects an element as a pivot and rearranges the array so that smaller elements are placed before the pivot and larger elements are placed after it. The algorithm then recursively sorts the elements on both sides of the pivot. This process continues until the subarrays contain zero or one element. Quick Sort can be very efficient in practice, although its performance depends on how the pivot is selected.

3. Pseudocode:

QuickSort(array, low, high)

 if low < high

 pivotIndex = Partition(array, low, high)

 QuickSort(array, low, pivotIndex - 1)

 QuickSort(array, pivotIndex + 1, high)

Partition

Partition(array, low, high)

```

pivot = array[high]

i = low - 1
for j = low to high - 1
    if array[j] <= pivot
        i = i + 1
        swap array[i] and array[j]

swap array[i + 1] and array[high]
return i + 1

```

4. Divide-and-Conquer:

Divide: Choose a pivot and partition the array around it.

Conquer: Recursively sort the elements before and after the pivot.

Combine: The sorted partitions together form the sorted array.

5. Base Case: The base case occurs when the subarray has zero or one element. It is already sorted.

6. Recursive Case: When the subarray has more than one element, Quick Sort partitions it around a pivot and recursively sorts the two resulting subarrays.

7. Time Complexity:

CASE	COMPLEXITY
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$

8. Space Complexity: $O(\log n)$ average case for the recursion stack and $O(n)$ worst case when the recursion becomes highly unbalanced.

1. Algorithm Name: Binary Search (Recursive)

2. Description:

Binary Search is used to find a value in a sorted list by repeatedly dividing the search area in half. It compares the target value with the middle element of the current search range. If the target is smaller, the search continues in the left half, and if it is larger, the search continues in the right half. This process continues recursively until

the target is found or the search range becomes empty. Binary Search is efficient for sorted data because it eliminates half of the remaining elements at each step.

3. Pseudocode:

BinarySearch(array, low, high, target)

if low > high

return -1

mid = (low + high) / 2

if array[mid] == target

return mid

if target < array[mid]

return BinarySearch(array, low, mid - 1, target)

else

return BinarySearch(array, mid + 1, high, target)

4. Divide-and-Conquer:

Divide: Find the middle element and divide the search range into two halves.

Conquer: Recursively search only the half that may contain the target.

Combine: No actual combining is required; the result is returned from the recursive call.

5. Base Case:

There are two base cases:

- The target is found: array[mid] == target
- The search range is empty: low > high

6. Recursive Case:

If the target is not the middle element, the algorithm recursively searches either the left half or the right half.

7. Time Complexity:

CASE	COMPLEXITY
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

8. Space Complexity: $O(\log n)$

Because the recursive calls create a recursion stack. In the worst case, there can be approximately $\log n$ recursive calls.

Part 2:

Problem Statement: Create your own list of 10,000 random integers. Your task is to:

1. Implement:

- Bubble Sort ($O(n^2)$)
- Merge Sort (recursive, $O(n \log n)$)
- Binary Search (recursive)

2. Measure and compare:

- Execution time for Bubble Sort vs Merge Sort on the same dataset.
- Execution time for searching a value using Linear Search vs Binary Search (on a sorted list).

3. Demonstrate the following Collections class methods:

- `Collections.sort()` – Sort the list and compare performance with your Merge Sort
- `Collections.binarySearch()` – Search for a value and compare with your Binary Search.
- `Collections.reverse()` – Reverse the sorted list.
- `Collections.shuffle()` – Randomize the list order.

Requirements:

- Use `System.nanoTime()` or `System.currentTimeMillis()` for timing.
- Print results in a comparison table.
- Explain why the performance differences occur.

```
Assignment1Task3...
168 ~ while (j < right.size()) {
169     list.set(k, right.get(j));
170     j++;
171     k++;
172 }
173 }
174 // LINEAR SEARCH
175
176 ~ public static int linearSearch(List<Integer> list,int target) {
177 ~ for (int i = 0; i < list.size(); i++) {
178 ~ if (list.get(i) == target) {
179     return i;
180 }
181 }
182 return -1;
```

```
Generated 10000 random integers.

Bubble Sort time: 724025611 ns
Merge Sort time: 15233963 ns

Search value: 49826
Linear Search index: 5000
Linear Search time: 670104 ns
Binary Search index: 5000
Binary Search time: 7519 ns

Collections.sort() time: 12162442 ns
Collections.binarySearch() index: 5000
Collections.binarySearch() time: 34942 ns

Collections.reverse() completed.
Collections.shuffle() completed.

COMPARISON TABLE
Bubble Sort           : 724025611 ns
Merge Sort            : 15233963 ns
Collections.sort()    : 12162442 ns
Linear Search         : 670104 ns
Binary Search         : 7519 ns
Collections.binarySearch() : 34942 ns

...Program finished with exit code 0
Press ENTER to exit console.
```

Performance Comparison: Bubble Sort is slower for 10,000 elements because its average and worst-case time complexity is $O(n^2)$. Merge Sort is faster because its time complexity is $O(n \log n)$. Collections.sort() is also efficient because Java uses an optimized sorting implementation. Linear Search can take $O(n)$ time because it may check many elements, while Binary Search takes $O(\log n)$ time because it eliminates half of the search space at each step. The execution times may vary depending on the computer and system condition.